



OpenFGA validates a JWT against an OIDC issuer's JWKS. (Dex is in OIDC mode, and OpenFGA is also running in OIDC mode). explain in detail the entire flow from LDAP to the Libcloud REST API all the security verification involved and how JWT played its part. Describe in point form to emphasize the chronological flow. emphasize on the type of security verification and how potentially it can be misused/broken.

Good — I have deep source material now. Let me compose this as a detailed, chronological point-form breakdown.

Here is the complete end-to-end flow with every security verification step and its associated attack surface.

Architecture Components in This Flow

- **LDAP** — identity store (users + groups)
- **Dex** — OIDC provider; uses LDAP as its LDAP connector; issues signed JWTs
- **libcloud REST API** — the API gateway; validates the JWT, then calls OpenFGA
- **OpenFGA** — authorization engine; validates a separate JWT (its own audience) before processing check requests
- **Vault** — secrets backend; uses the Vault LDAP auth method to verify group membership and issue a Vault token for credential fetching

Phase 1 — User Authentication: Browser/Client → Dex → LDAP

Step 1.1 — Client initiates OIDC Authorization Code Flow

- The user's client (CLI tool, portal, or script) sends an OAuth2 authorization request to Dex's /auth endpoint
- **Parameters:** response_type=code, client_id=libcloud-rest, scope=openid profile email groups, redirect_uri, state (CSRF token), nonce (replay prevention)

- **⚠ Misuse:** If `state` is not validated on callback, an attacker can perform a CSRF attack — substituting their authorization code for the victim's, hijacking the session

Step 1.2 — Dex redirects user to login form

- Dex presents credentials form (username + password)
- **⚠ Misuse:** If Dex is served over HTTP (not HTTPS), credentials are transmitted in cleartext and trivially intercepted

Step 1.3 — Dex performs LDAP bind against LLDAP

- Dex uses its configured service account (`bindDN`) to connect to LLDAP over LDAPS
- Dex executes a **user search** to find the user's DN (e.g. `uid=alice,ou=people,dc=example,dc=com`)
- Dex then performs a **user bind** using the submitted password to verify it
- **⚠ Misuse:** If LDAP is plain (not LDAPS), the bind password is sniffable on the network — **wire-level credential interception**
- **⚠ Misuse:** If the Dex service account has write permissions to LLDAP, a Dex compromise means an attacker can modify group memberships — **privilege escalation via upstream manipulation**

Step 1.4 — Dex performs group membership LDAP search

- Dex searches for all groups the user is a member of (e.g. `memberOf` attribute or `member` filter against group OUs)
- The groups claim requires the groups scope to be included in the authorization request^[1]
- Result: a list of group names e.g. `["cloud-admin-aws", "cloud-ro-gcp"]`
- **⚠ Misuse:** Dex caches group membership for the lifetime of the refresh token — **stale group claims** mean a user removed from a group in LLDAP still holds a valid token asserting the old membership until expiry^[2]
- **⚠ Misuse:** If group names are not normalised and validated, an attacker who can create an LLDAP group with a carefully chosen name can inject themselves into a role simply by being added to that group

Step 1.5 — Dex issues authorization code

- Dex returns an authorization code to the client via the `redirect_uri`
- The `state` parameter is echoed back for CSRF verification

Phase 2 — Token Issuance: Client → Dex Token Endpoint

Step 2.1 — Client exchanges authorization code for tokens

- Client sends POST `/token` to Dex with: `grant_type=authorization_code`, `code`, `redirect_uri`, `client_id`, `client_secret`

- **⚠ Misuse:** If `client_secret` is weak or stored in plaintext in the client application, an attacker can impersonate the client and exchange stolen codes

Step 2.2 — Dex constructs and signs the ID Token (JWT)

- Dex assembles the JWT payload with claims including: ^[1]

```
{
  "iss": "https://dex.example.com",
  "sub": "CgVhbGljZRIEbGRhcA",
  "aud": "libcloud-rest",
  "exp": 1751500000,
  "iat": 1751496400,
  "nonce": "<nonce from step 1.1>",
  "email": "alice@example.com",
  "name": "Alice",
  "groups": ["cloud-admin-aws", "cloud-ro-gcp"],
  "federated_claims": { "connector_id": "ldap", "user_id": "alice" }
}
```

- Dex signs this JWT using its **private RSA or ECDSA key** (RS256 or ES256)
- The corresponding public key is published at `https://dex.example.com/keys` (JWKS endpoint)
- **⚠ Misuse — `alg:none` attack:** If a relying party does not pin the expected algorithm and a library naively trusts the JWT header's `alg` field, an attacker can strip the signature and set `alg:none` to forge arbitrary claims ^[3]
- **⚠ Misuse — RS256→HS256 confusion:** If a verifier accepts HS256 when RS256 is expected, the attacker signs a forged token using the **public key as the HMAC secret** — public keys are by definition public ^[3]
- **⚠ Misuse — `jku/jwk` header injection:** Some JWT libraries fetch the key from a URL in the JWT header's `jku` field or accept an embedded `jwk` field. An attacker hosts their own JWKS and signs a token with their own private key ^[3]
- **⚠ Misuse — weak signing key:** If Dex is misconfigured with a short or guessable HMAC secret, the token can be cracked offline with hashcat (`-m 16500`) and any claim forged ^[3]

Step 2.3 — Client receives `id_token`, `access_token`, `refresh_token`

- The `id_token` is the JWT used by libcloud REST
- The `refresh_token` allows the client to get new tokens without re-authenticating
- **⚠ Misuse:** Refresh tokens are long-lived. If stored insecurely (browser `localStorage`, unencrypted disk), they persist across sessions and allow token renewal long after the user intended to log out
- **⚠ Misuse — token replay:** A stolen `id_token` within its `exp` window can be replayed to any service that accepts it. Dex does not issue `jti` claims by default, so per-token revocation is not natively possible without an external blocklist ^{[4] [5]}

Phase 3 — libcloud REST API: JWT Verification (Authentication)

Step 3.1 — Client sends request with JWT as Bearer token

```
POST /compute/aws/nodes
Authorization: Bearer <id_token>
```

Step 3.2 — libcloud REST fetches Dex's JWKS

- On first request (or on key rotation), libcloud REST calls `https://dex.example.com/keys` to retrieve the current JWKS
- The response contains one or more RSA/EC public keys, each identified by a `kid` (Key ID)
- libcloud REST caches the JWKS with a TTL (typically the `Cache-Control` value from Dex)
- ⚠ **Misuse — JWKS cache poisoning:** If the JWKS is fetched over HTTP or without certificate pinning, a MITM attacker injects a malicious JWKS, causing libcloud REST to trust tokens signed by the attacker's key
- ⚠ **Misuse — stale cache after key rotation:** If the JWKS cache TTL is too long and Dex rotates its signing key, legitimate tokens signed with the new key are rejected. Worse, if the cache is not invalidated on `kid` miss, the system may break or fall back insecurely

Step 3.3 — libcloud REST verifies the JWT

The following checks must all pass:

Check	JWT Field	Failure Consequence
Signature valid	Header <code>kid</code> → JWKS lookup → verify	Forged token accepted if skipped
Algorithm is expected (RS256/ES256)	Header <code>alg</code>	<code>alg:none</code> or confusion attack if not pinned
Issuer matches configured Dex URL	<code>iss</code>	Token from a foreign IdP accepted
Audience matches this service	<code>aud == "libcloud-rest"</code>	Confused deputy: token for another service accepted
Token not expired	<code>exp > now()</code>	Replayed expired token accepted
Token not used before valid time	<code>nbf <= now()</code>	Pre-issued token used prematurely
Nonce matches (if login flow)	<code>nonce</code>	Replay of a prior login session

- ⚠ **Misuse — aud bypass:** The single most exploitable misconfiguration. If `aud` is not validated, a token issued for a different Dex client (e.g. a monitoring tool) can authenticate to libcloud REST. This is a known real-world CVE pattern — CVE-2026-55689 in OpenFGA itself documents exactly this: audience validation was skipped when `--authn-oidc-audience` was not set ^[6] ^[7]

Step 3.4 — libcloud REST extracts identity from claims

- `sub` or `email` → username for logging

- groups → the user's LLDAP-derived role list
- This identity context is passed to OpenFGA in the next phase
- **⚠ Misuse — claim injection via group name crafting:** If libcloud REST passes the groups claim directly to OpenFGA as the user object without sanitisation, an attacker who controls their own LLDAP group name could inject a group value that matches a high-privilege OpenFGA relation (e.g. a group named `role:cloud-owner` if the prefix is not enforced)

Phase 4 — OpenFGA Authorization Check

Step 4.1 — libcloud REST authenticates to OpenFGA with its own JWT

- libcloud REST acts as an **OpenFGA client**, not as the end user
- It must present its own service JWT to OpenFGA (obtained from Dex using a `client_credentials` grant for the `libcloud-rest-service` OIDC client)
- OpenFGA is configured in OIDC mode: `authn.method=oidc`, `authn.oidc.issuer=https://dex.example.com`, `authn.oidc.audience=openfga`^[8]
- **⚠ Misuse — missing aud on OpenFGA:** Prior to OpenFGA v1.18.0, if `--authn-oidc-audience` was not set, any token from the same Dex issuer — including the end user's `id_token` — could authenticate directly to the OpenFGA API. Fixed in v1.18.0 which refuses to start in OIDC mode without both issuer and audience configured^[7] ^[6]

Step 4.2 — OpenFGA verifies the service JWT

- OpenFGA fetches Dex's JWKS from `https://dex.example.com/keys`
- Performs the same verification chain as Step 3.3 but for the service token with `aud=openfga`^[9]
- **⚠ Misuse:** If the `aud` claim checking is bypassed (misconfigured or old version), any valid Dex-issued JWT authenticates to OpenFGA, allowing direct tuple manipulation without going through libcloud REST

Step 4.3 — libcloud REST sends an OpenFGA Check request

```
{
  "tuple_key": {
    "user": "user:alice",
    "relation": "can_write",
    "object": "api:compute.provision"
  },
  "context": {
    "provider": "aws"
  }
}
```

- The `user` field is populated from the end user's JWT `sub` or `email` claim
- **⚠ Misuse — user field spoofing:** If libcloud REST is compromised or misconfigured and constructs the `user` field from an attacker-controlled input rather than the verified JWT

claim, the attacker can check or act as any user

Step 4.4 — OpenFGA evaluates the authorization model

- Traverses the relationship graph: `user:alice` → member of `role:cloud-admin-aws` → `can_write` on `api:compute.provision`
- Returns `allowed: true` or `allowed: false`
- ⚠ **Misuse — tuple poisoning:** If an attacker can write tuples directly to OpenFGA (by obtaining the service JWT or exploiting a missing audit check), they can grant themselves arbitrary permissions without going through LLDAP

Step 4.5 — libcloud REST enforces the decision

- `allowed: false` → return HTTP 403 immediately, do not proceed
- `allowed: true` → proceed to Vault credential fetch
- ⚠ **Misuse — TOCTOU (Time-of-Check to Time-of-Use):** In a high-concurrency scenario, a role could be revoked in LLDAP and the tuple deleted in OpenFGA between the Check call and the actual cloud API call. libcloud REST should minimise this window — ideally the Check and the Vault fetch happen in the same transaction with no delay

Phase 5 — Vault Credential Fetch

Step 5.1 — libcloud REST authenticates to Vault

- Uses Vault's LDAP auth method: presents the user's username + a service credential, or uses the Vault token obtained earlier from the LDAP bind
- Vault performs a live LDAP bind against LLDAP/AD to verify the user's group membership
- Maps group → Vault policy (e.g. `cloud-admin-aws` → `cloud-admin-aws-policy`)
- ⚠ **Misuse — Vault LDAP auth bypass:** If the Vault LDAP auth method is configured with a weak or shared bindpass for its service account, an attacker who obtains that password can enumerate Vault policies without being a real user

Step 5.2 — Vault issues a short-lived token scoped to the user's policies

- Token TTL is typically 1–4 hours
- The token is bound to the policies matching the user's LLDAP groups
- ⚠ **Misuse — token leakage in logs:** If Vault tokens appear in libcloud REST application logs (e.g. logged as part of HTTP headers in debug mode), a leaked log file yields valid Vault tokens

Step 5.3 — libcloud REST reads dynamic cloud credentials

- Calls e.g. `vault read aws/creds/ec2-admin` with the scoped token
- Vault's AWS secrets engine calls AWS STS, creates a time-limited IAM credential, and returns it

- **⚠ Misuse – credential exfiltration:** The dynamic credential is plaintext in the `vault read` response. If this response is logged or stored, it is a live cloud credential. Even though it is time-limited, within the `lease_duration` it grants cloud API access with no further checks

Step 5.4 – libcloud REST executes the cloud API call

- Uses the dynamic AWS/GCP/Azure credential to call the provider API
- The credential is discarded after use (not stored by libcloud REST)
- **⚠ Misuse – credential scope too broad:** If the Vault role's IAM policy is overly permissive (e.g. Action: "*"), a compromised libcloud REST instance or a stolen credential gives full cloud access regardless of what OpenFGA authorized

Phase 6 – Token Expiry, Refresh, and Revocation

Step 6.1 – JWT expiry and refresh

- The `id_token` has a short TTL (typically 1 hour in Dex)
- The client uses the `refresh_token` to obtain a new `id_token` from Dex's `/token` endpoint
- Dex re-queries LLDAP for group membership on refresh – this is when revoked group access takes effect^[2]
- **⚠ Misuse – refresh token theft:** A stolen refresh token allows indefinite token renewal until: (a) the refresh token itself expires, (b) Dex's session is revoked server-side, or (c) the user's LLDAP account is disabled. There is no per-token revocation in standard OIDC without a revocation endpoint

Step 6.2 – Vault lease expiry

- Dynamic cloud credentials expire at `lease_duration`
- Non-renewable leases simply expire; renewable leases can be extended up to `max_ttl`^[10]
- **⚠ Misuse – lease renewal as persistence:** An attacker with a valid Vault token can repeatedly renew leases to extend credential lifetime indefinitely as long as the token itself is valid

Summary: Security Verification Chain and Failure Points

LLDAP —(LDAPS bind)→ Dex —(RS256 signed JWT)→ libcloud REST

Verifies: sig, alg, iss,
aud, exp, nbf, nonce

OpenFGA (service JWT + Check)
Verifies: sig, iss, aud
Evaluates: relationship graph

Vault (LDAP bind → token)
Verifies: group membership
Issues: scoped dynamic creds

The two highest-severity misconfigurations across this flow are:

1. **Missing aud validation** — at libcloud REST (accepts cross-service tokens) and at OpenFGA (CVE-2026-55689) — this single omission collapses the entire boundary between services^[6]
^[7]
2. **Stale groups claim in JWT** — a user removed from LLDAP still holds a valid token with the old group membership until expiry; the only reliable revocation path is short token TTLs (≤15 min) combined with Dex session revocation^[2]

✱✱

1. <https://dexidp.io/docs/configuration/custom-scopes-claims-clients/>
2. <https://dexidp.io/docs/connectors/oidc/>
3. <https://bipi.in/blog/jwt-attacks-alg-none-kid-jwk-cracking>
4. <https://elsyarifx.medium.com/the-hidden-power-of-jti-how-a-single-claim-can-stop-token-replay-attacks-0255fbcf6b9b>
5. <https://stackoverflow.com/questions/28907831/how-to-use-jti-claim-in-a-jwt>
6. <https://corgea.com/advisories/vulnerabilities/CVE-2026-55689>
7. <https://github.com/advisories/GHSA-hcxc-wf8j-23hv>
8. <https://openfga.dev/docs/getting-started/setup-openfga/configure-openfga>
9. <https://medium.com/@torinks/keycloak-and-aud-claim-usage-as-an-additional-authorization-layer-3e0ab921e569>
10. <https://developer.hashicorp.com/vault/docs/concepts/lease>
11. <https://github.com/dexidp/dex>
12. <https://www.openpolicyagent.org/docs/oauth-oidc>
13. <https://github.com/oauth2-proxy/oauth2-proxy/issues/1588>
14. https://zenn.dev/suwash/articles/keycloak-oidc-kong-openfga-react-spa_20260404
15. <https://blogs.businesscompassllc.com/2026/06/amazon-eks-dashboard-security.html>
16. <https://medium.com/upstream-engineering/kubernetes-authentication-using-ldap-and-oauth2-83c3457becf8>
17. https://support.huaweicloud.com/intl/en-us/my-kualalumpur-1-usermanual-cce/cce_10_0997.html
18. <https://github.com/banzaicloud/jwt-to-rbac/blob/master/README.md>
19. <https://github.com/dexidp/dex/issues/1260>
20. <https://hasura.io/docs/2.0/enterprise/sso/ldap/>
21. <https://www.youtube.com/watch?v=5fu6g1o-WzM>
22. <https://github.com/dexidp/dex/issues/2657>

23. <https://medium.com/trendyol-tech/kubernetes-authentication-and-authorization-through-dex-ldap-and-rbac-rules-c2e03111b408>
24. <https://medium.com/@shubhamatucsd/the-complete-guide-to-oauth-2-0-openid-connect-and-jwt-token-verification-f2516c196f3b>
25. <https://openfga.dev/docs/modeling/token-claims-contextual-tuples>
26. <https://openfga.dev/docs/getting-started/setup-openfga/access-control>
27. <https://github.com/dexidp/dex/issues/432>
28. <https://libraries.io/go/github.com%2Fconcourse%2Fdex%2Fexamples>
29. <https://github.com/Hawxy/Fga.Net>
30. https://dev.to/genius_introuble/advanced-jwt-exploitation-techniques-going-beyond-the-basics-1h4m
31. <https://github.com/banzaicloud/jwt-to-rbac>
32. <https://codingtechroom.com/question/preventing-replay-attacks-in-jwt-authentication-proper-use-of-jti-claims>
33. <https://stackoverflow.com/questions/44658963/how-to-properly-use-jti-claims-with-jwt-to-prevent-replay-attacks>
34. <https://stackoverflow.com/questions/52281001/jwt-replay-validation-based-on-jti-claim-rather-than-on-expiration-time>
35. <https://dexidp.io/docs/connectors/microsoft/>
36. <https://www.youtube.com/watch?v=hCmmhRT0zbQ>
37. <https://github.com/jitsi/jitsi-meet/issues/16446>
38. <https://www.shop.bottegadelsarto.com/feed/jwt-jti-claim-explanation-finally-makes-this-click-272554>
39. <https://dexidp.io/docs/guides/kubernetes/>
40. <https://advisories.gitlab.com/golang/github.com/openfga/openfga/CVE-2026-55689/>